Design of AI Systems, Assignment 3
AI Tools

Tim Bakkenes & Anton Johansson
Group 54

February 14, 2025

1 Introduction

This is a report for the third assignment in the Design of AI Systems course at Chalmers. In this assignment, the goal is to read about and experiment with the deployment of AI tools.

The code for this assignment can be found at the end of the document in the appendix and explained in the respective parts of the report. In order to learn more about the implementation of this algorithm there was no use of the KMeans library provided my Scikit-learn. This choice made us think extra about the data as it flows through the model and about the different aspects of the model and the data that influence the final result. In the end, the result was a function KMeans classifier with many different tunable hyperparameters and variations.

2 Reading and reflection

In this part of the assignment, the paper *Hidden Technical Debt in Machine Learning Systems*[10], written by employees from Google, was supposed to be read and summarized and the key takeaways should be highlighted. There are a lot of good material and takeaways from the article and some of those will be covered below.

2.1 Summary of Article

The article [10] is an article about *technical debt* and machine learning systems. Using the software engineering framework of technical debt, the authors of [10] explore various ML-specific issues that cause massive ongoing maintenance costs and how to combat them.

What makes this topic especially important for machine learning systems is that they have a "special capacity for incurring technical debt" according to [10] because besides from suffering from the same maintenance problems of traditional code, ML systems also suffer from ML-specific issues[10]. The main problems that are being addressed by [10] are connected to the entanglement of modules via different kinds of dependencies such as consumers, data dependencies, configurations and a few other issues. Traditionally in software engineering, strong abstraction boundaries and a modular design help create maintainable code with fewer dependencies which allows isolated changes to be made. However, for machine learning systems it is difficult to do this according to [10] since the behavior of the models doesn't only rely on software logic.

Machine learning systems can inherently create more entanglement and make isolated improvements impossible[10]. [10] mentions how changing the input distribution of one feature signal will impact the importance,

weights or use of the other features. This is a key take-away from [10] and is referred to as the CACE principle: Changing Anything Changes Everything. The principle applies to essentially every possible tweak of a model and is important to keep in mind when developing machine learning systems[10].

Diverse and ever changing data, referred to as unstable input signals by [10], is a common in machine learning and can be harmful to consuming systems, even if the change was an improvement. This is also key takeaway to remember, it is important to carefully consider the effects of a change to the consumers of or in that system. There are a lot of other factors to consider that are impacted by changes in the data or model. Some issues mentioned are models that depend on each other, hidden consumers that depend on models and models that depend on data [10]. In these cases the cost and difficulty of changing the model will be significantly higher, as changing it can have undesirable side-effects on other systems[10]. This makes it much harder to make changes and improvements to individual models or data flow.

[10] proposes many strategies to mitigate and avoid the situations discussed above. For example, to enforce access restrictions on models, accepting the cost of making a separate model to avoid correction cascades and to create versioned copies of input data[10]. More issues and strategies are highlighted but the maybe biggest takeaway solution is that creating a team culture that rewards improving the maintenance of ML systems is very important for long term successful development.

3 Classifier Implementation

3.1 Exploratory Data Analysis

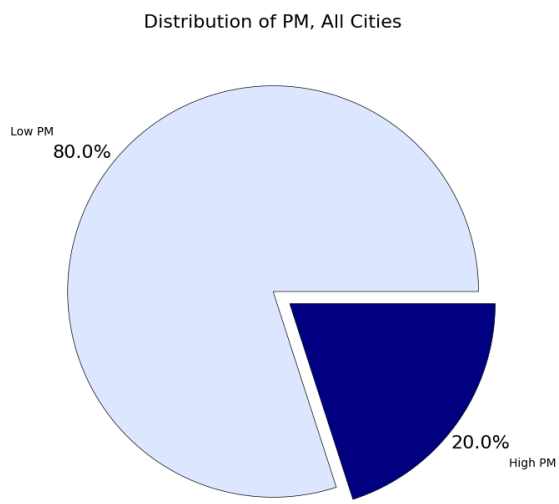


Figure 1: Distribution of PM for All Cities

City	Low PM
Beijing	0.687
Chengdu	0.705
Guangzhu	0.936
Shanghai	0.902
Shenyang	0.822

Table 1: Distribution of PM by city

Figure 1 and Table 1, it can be seen that 80% of all days had a High PM. 20% is a good distribution of positive values for the target feature. This increases the possibility for a successful prediction model, compared to if the positives were only 0.1% for example. A balanced data distribution is particularly important for k-means since its performance tends to be affected by skewed data [8]. In this case, the moderately balanced distribution should mean that k-means will work pretty well for this data. And if the performance is mediocre, using techniques to make the dataset more balanced might help to improve performance.

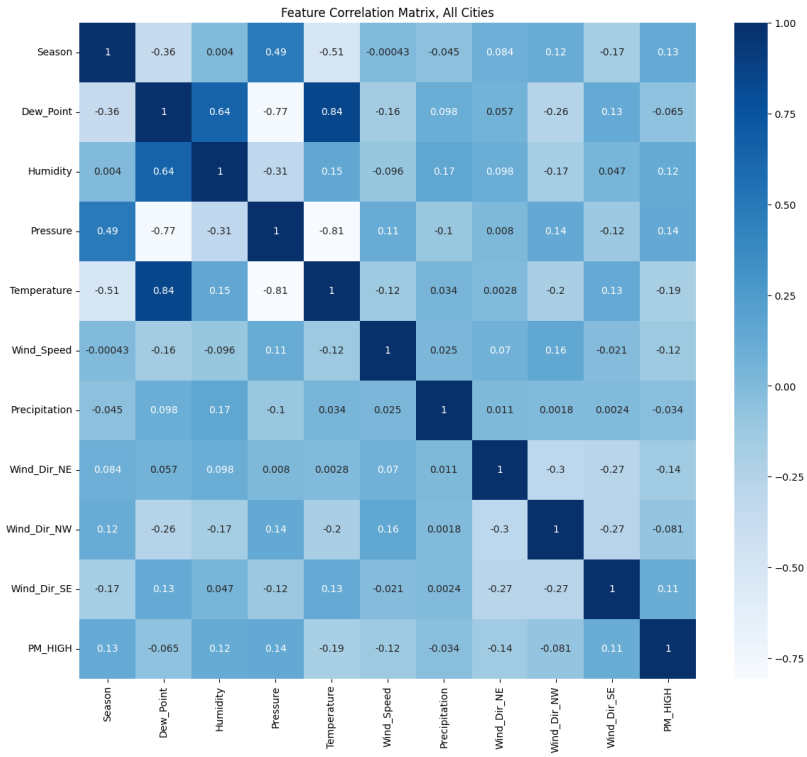


Figure 2: Feature Correlation Matrix for All Cities

The correlation matrix is a popular tool to quickly get a view of how every feature correlates. Correlations mean how much two features "move" in the same direction. The diagonal is all ones as expected since the features are 100% correlated with themselves. Apart from that, we can see some clear relations between temperature and dew point for example, which makes a lot of sense and indicates the validity of the data. Regarding the correlations for PM high, there are multiple somewhat strong correlations. This suggests that the features should hold enough "value" to predict PM high pretty accurately.

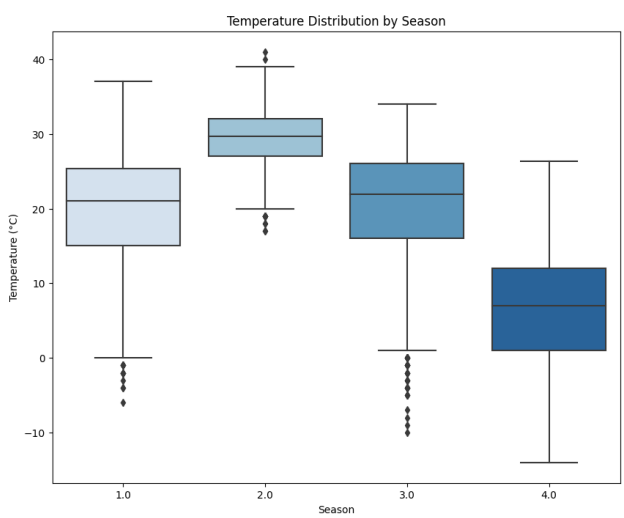


Figure 3: Boxplot grouped by Seasion for All Cities

Regarding the boxplot for all cities, a clear seasonal temperature pattern can be observed. This indicates that the data is accurate since the temperature indeed variates over the seasons in these cities.

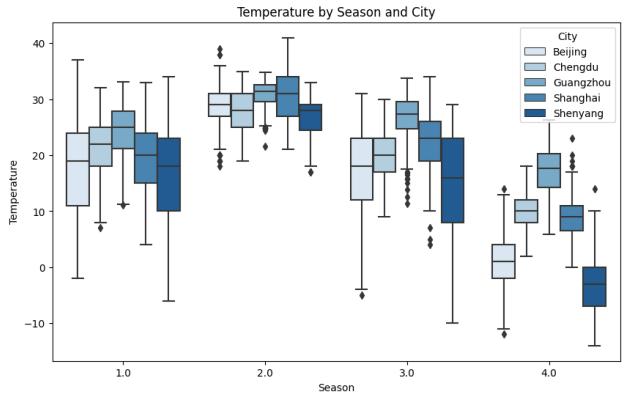


Figure 4: Temperature by Season and City

The more detailed plot in Figure 4 shows that the cities has some differences in terms of temperature. This will lead to more uncertainties for the model when predicting new cities.

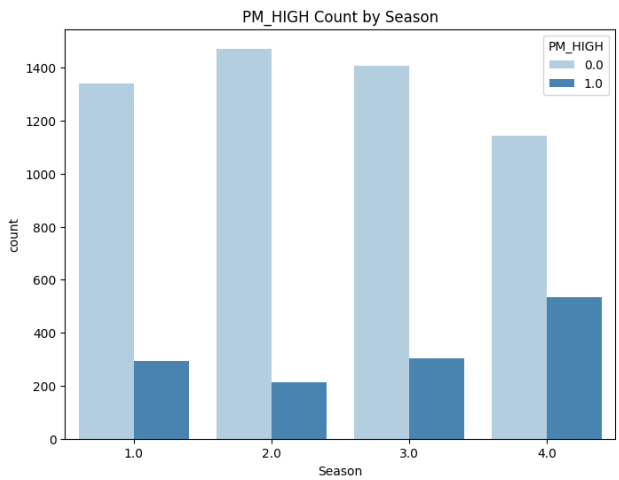


Figure 5: PM high Count by Season

Figure 5 shows more clearly that the season is correlated with PM high, to some degree.

Feature	Count	Mean	Min	Max
Season	6708	2.51	1	4
Dew_Point	6708	8.24	-36	28
Humidity	6708	54.03	3	100
Pressure	6708	1014.05	987.6	1044
Temperature	6708	19.02	-14	41
Wind_Speed	6708	21.27	0	617
Precipitation	6708	0.13	0	44.4
Wind_Dir_NE	6708	0.23	0	1
Wind_Dir_NW	6708	0.23	0	1
Wind_Dir_SE	6708	0.20	0	1
PM_HIGH	6708	0.20	0	1

Table 2: Statistics for All Cities

From Table 2 it can be observed that:

- Season ranges from 1-4, and the mean is 2.5 meaning that seasons from 1-4 are likely evenly distributed.
- Dev point seems reasonable.
- Humidity seems reasonable.
- Pressure seems reasonable.
- Temperature seems reasonable.
- Wind speed has a max value of 617 m/s. At first glance this looks like an outlier because it is ten times higher than the maximum threshold of hurricanes according to [6]. However this measurement represents the *Cumulated wind speed (m/s)* as stated in the data card [7], meaning that likely is the sum of the wind during one day. In this case, the data is reasonable.
- Precipitation seems reasonable.
- Wind direction is binary and seems reasonable.
- PM is binary and seems reasonable.

3.2 Train and Test Split

In the assignment description, the idea is to train and validate the classifier on the cities Beijing and Shenyang and then evaluate it separately on the cities Guangzhou and Shanghai. Below is an description of why it is important to split the data into a training set and a testing set.

The dataset was split into a training set and testing set. By doing this, it is possible to test the model on, what to the model is, unseen data. If the same data is used for training and testing the models, the testing results would not be a reliable metric of the predictive power of the model. A complex model would in that case be able to memorize the data and be able to generate a very good accuracy when tested. And when the model later is tested on unseen data it could generate worse results.

3.3 Benchmark

Two methods were used as benchmarks. The first one is to just select the majority class which means that if more than 50% have the label 1 then that label will be predicted every time. The goal would be to have a higher accuracy than the one achieved by this one.

3.4 The theory of K-means

According to [5], the K-means algorithm is an iterative clustering algorithm that partitions the data into groups based on the distance to centroids. The number of centroids is a hyperparameter that is specified to the model. Each centroid can be assigned a label that is usually the majority class of that cluster, and then the predictions of all points that are closest to a certain centroid is assigned that label as well.

The first step in the algorithm for K-means is to initialize K centroids[5]. This can be done randomly or by using some other sampling method[5]. The next step is the iterative part of the algorithm and for each iteration, two things are done[5]. First, each data point is assigned to the closest centroid in terms of distance, usually euclidean distance[5]. The euclidean is simply the shortest path between two points, A and B, and the formula for those points if they are in a n dimensional space is:

$$Euclidian_distance(A, B) = \sqrt{\sum_{i=1}^{n}(A_i - B_i)^2}$$

The next step is to calculate the mean of all the points in each cluster and then positions the centroid of that cluster at that mean[5]. The mean can be understood as the center in a way. This process will continue either until convergence or until the maximum number of iterations has been reached[5].

Something important that is highlighted by [5] is that K-means clustering is very sensitive to the initial conditions, how the centroids are initialized and also sensitive to outliers. It is therefore important to optimize the centroid initialization and the hyperparameter K that explains the number of centroids.

Then once the clustering has been done, each centroid is assigned the label of the majority class among all training data points that belong to it. To then make predictions, the label of the centroid that is closest to the x the prediction is to be made from will be predicted.

K-means ++
One way to improve the initialization of the centroids is called the K-means ++ algorithm and is explained by [5]. It works by first choosing a single centroid from the dataset and then for each subsequent centroid, the distance from each point in the data to its nearest cluster center is calculated[5]. The next centroid is selected by looking at the likelihood that a point is at a proportional distance from the nearest centroid that was selected earlier. This algorithm is run iteratively until all the centroids has been placed.

The Elbow Method
To figure out the best value of K, the elbow method can be used[5]. It is a graphical method for finding the best value for K and works by measuring the euclidean distance between each data point and its centroid[5]. The number of clusters are then chosen based on when the "within cluster sum of squares" curve starts to flatten out[5]. The following implementation was implemented as shown in [2]. In [2], distortion is used as the metric which is the average squared distance from each point to its cluster center.

Convergence
According to [4] a convergence criterion for K-means that can be used is to look at how much the clusters change during an iteration. For example, if the criterion is set to 0.02, 2%, the algorithm is said to have converged if one iteration does not move any cluster centers by more than 2% of the smallest distance between any initial clusters[4].

3.5 Implementing K-means

The K-means algorithm explained above is a rather simple algorithm which naive implementation does not require a lot of code. The version implemented for this assignment follows the algorithm outlined in [5] by utilizing various Numpy functions to make it more efficient. In order to make it more interesting than the simplest implementation two different methods for initializing the centroids will be experimented with. The first one being to randomly initialize the centroids and the second is to implement the so called K-means ++ initialization method that is explained in the section before.

In order to randomly initialism the centroids, the pandas function `sample()` was used in the following way: `X.sample(n=self.number_of_clusters)`, where X is the input data and n is the number of centroids to select.

As explained in the section about the theory of K-means the K-means ++ algorithm's way of initializing the centroids is more complex. But the first step is to randomly select a centroid and it can be done in the same way as before: `X.sample(n=1)`. In order to figure out the rest of the algorithm, the source code of the Scikit-learn library's implementation of KMeans was used a lot for inspiration. The same steps were implemented with different methods and simplified functions. How it is implemented is by calculating the euclidean distances from each data-point to each other centroid ad them summing those values and use that value to describe something refereed to as potential in the source code. The equation that explains this is the following:

$$P = \sum_{i=0}^{N-1} \min_{p \in \{0,\ldots,k-1\}, p \leq a} \left(\sqrt{\sum_{j=0}^{n-1} (x_{ij} - c_p)^2} \right)$$

where a i the current cluster index being added, N the number of rows, n the number of features and c_p is the cluster that results in the shortest distance for that row. If the second centroid is being added, $a = 1$ and so for each data-point the distance to a cluster that is shortest distance away will be added to the potential. The algorithm will then generate a few random values and test which one will result in the biggest drop in potential, ΔP. The number of random values to test is a hyperparameter that can be specified, `n_local_trials` and a larger value will make it so more values are tried. This procedure of calculating the potential and seeing which random value minimize it continues until the specified number of centroids has been achieved.

After the centroids have been initialized, the algorithm follows the algorithm outlined in [5] as mentioned before and in section *3.4*.

One hyperparameter that can be specified is `max_iterations` and it basically specifies how many iterations the algorithm will run when fitting to a dataset. However, having another way to determine when convergence has been reached and terminate the process would be convenient. Therefore, the convergence idea proposed by [4] and discussed briefly n section *3.4* was also implemented. So if the largest euclidean movement of any one centroid is less than a factor multiplied by the minimum euclidean distance between two centroids from the previous iteration, the model has converged. In mathematical terms, the model has converged if the following statement holds:

$$max(centroid_movement) < \alpha * min_distance_between_centroids$$

where α is a hyperparameter called `convergence_threshold` that is a float value between 0 and 1. There is an edge case that presents itself when the number of clusters is equal to 1. Then the model would just automatically converge after the first iteration since there is no minimum distance between centroids. This is not a problem however, because when there is only one centroid the model will always only require one iterations since the mean of the cluster will be the best possible fit.

One other thing to point out is that an optional argument for the default scoring function used when calling `score()` was used. The idea was that a scoring function could be passed to the `score()` method of the **KMeansClassifier** class. And there is of course a random state parameter as well that can be added to make sure the results are reproducible.

4 Training and Validation

Now that the model has been implemented the next step is to train and validate it on the cities Beijing and Shenyang. One of the first ideas that came to mind was to look at the cross validation score for different values of K using F1-score and accuracy score, where accuracy score is defined as the fraction of predictions that are correct. However, this is not perfect. Consider a case when it looks like there are 2 clusters and

the hyperparameter for the number of clusters is 3. Two of the centroids will likely be in the same cluster and one might be useless. There is also the issue of overfitting, if the number of centroids is to large, the model runs the risk of fitting to outliers which can lead to poor generalization, especially in the case when the model is going to be tested on data from different cities later. That is because the classifier might adapt to some city specific features rather than capturing the overall patterns.

First of all, it was attempted below as can be seen in figure 6 to use F1-score as the scoring function and see how it changes as the number of clusters increase.

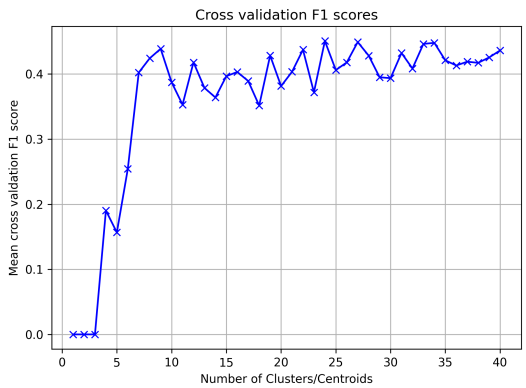


Figure 6: Cross validation F1 scores for different number of clusters

The same as above was also done with the normal accuracy score function and the elbow method was also used as explained in section *3.4*.

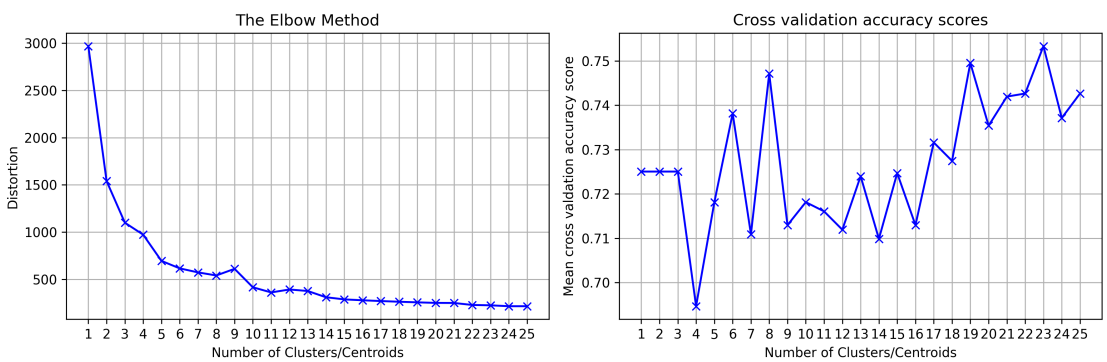


Figure 7: Elbow method and cross validation accuracy scores for different number of clusters

The same code as the one on the top can also be run for up to larger values of the number of clusters and that is shown below in figure 8. In this case the random state was not specified so the result will be different for when the program was run only up to 25 centroids.

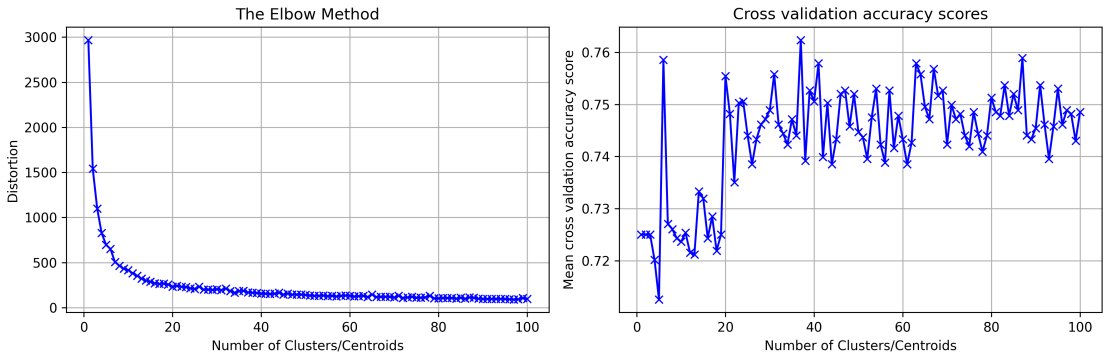


Figure 8: Elbow method and cross validation accuracy scores for different number of clusters, up to 100

It is interesting to see that the *elbow* seems to appear at different number of centroids depending on how far the algorithm is run. And since the curve does not appear to have a clear point where it bends more than the others perhaps the method is not ideal for this. If the result in figure 8 was used to choose the value of the number of clusters it would be set to rougly 8, which is also the default value for both KMeans and the implemented KMeansClassifier.

All of these tests were done using the KMeans ++ method of initializing the centroids since it in theory should be the better strategy and is the one used by the Scikit-learn implementation. However, during testing, there was no significant change between the two methods for this dataset as can be seen below in figure 9.

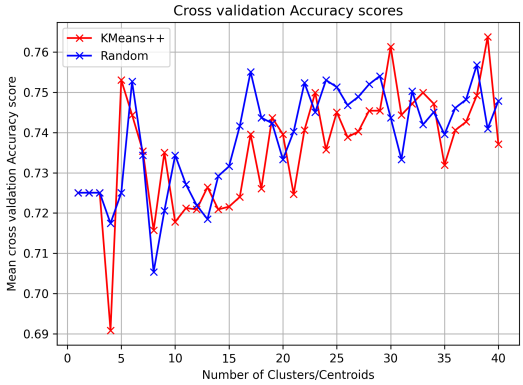


Figure 9: Random vs KMeans++ centroid initialization

The same can be seen when observing the F1-score:

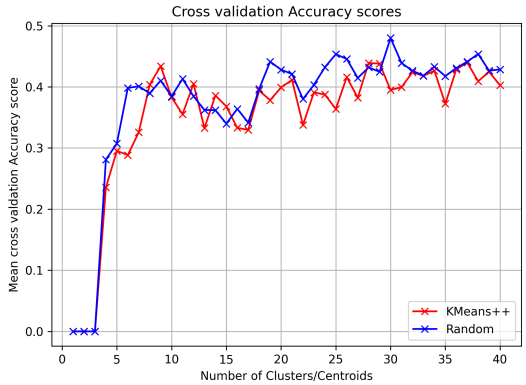


Figure 10: Random vs KMeans++ centroid initialization, F1-score

The hyperparameter for the number of local trials during KMeans ++ initialization did not seem to affect this behavior either.

The final training setup is shown below:

```
classifier = KMeansClassifier(number_of_clusters=8,
                              centroid_initialisation="k-means++",
                              max_iterations=1000,
                              random_state=42)

classifier.fit(X_train, y_train)
```

5 Evaluation and Results

In this part of the assignment the goal is to evaluate the classifier that has been experimented with and validated and trained in previous sections of the report. The classifier should now be evaluated on the cities Guangzhou and Shanghai. In order to do the evaluation, the model with the hyperparameters from the previous section was used after it had been trained on the training set using the `fit()` method. The data from the cities Guangzhou and Shanghai was loaded in and split into two variables, `X_test` and `y_test` where the first one is the features and the second one is the target labels. Furthermore, a **DummyClassifier** from Scikit-learn was imported and fitted to the training data to always predict the most common label. The next step was just to call the score function to get the results. They are shown below in figure 11.

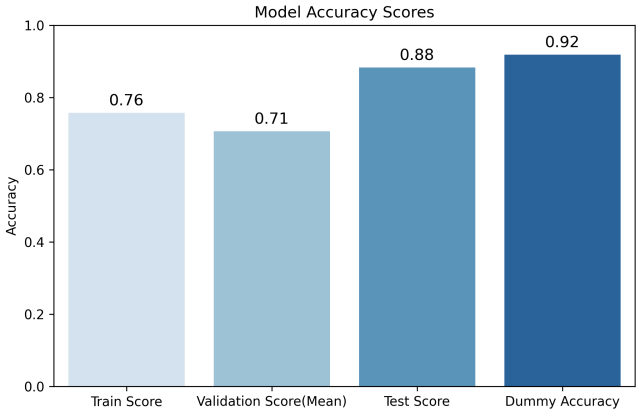
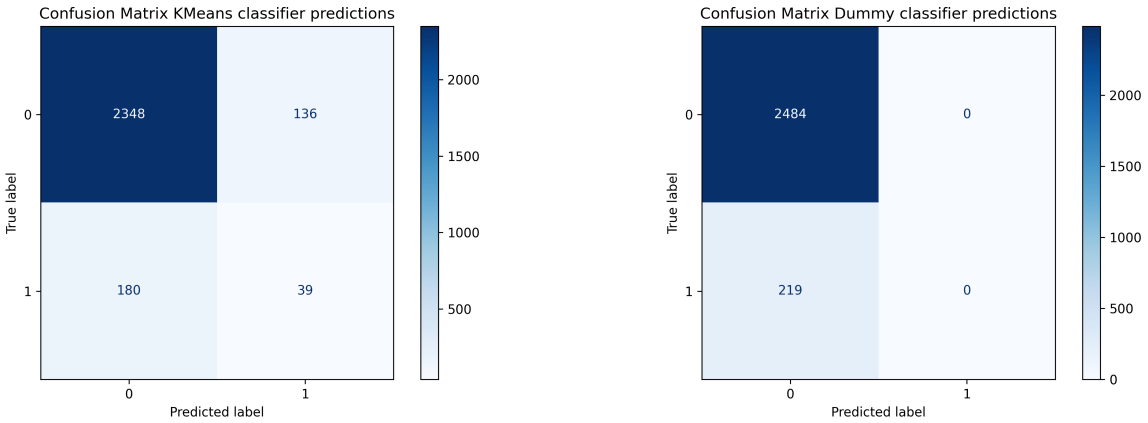


Figure 11: Accuracy Comparisons

When observing the accuracy score, the DummyClassifier actually performed better. It predicted only 0:s, that the "PM_HIGH" feature is 0, meaning that the concentration of particles called PM2.5, is lower than $100\mu g/m^3$. And since the values of the "PM_HIGH" for Guangzhu and Shanghai are 93.6% and 90.2% 0:s respectively the dummy classifier would naturally have an accuracy score of above 90%. Likewise, most of the data overall have a "PM_HIGH" value of 0 which means the KMeansClassifier is trained on primarily that kind of data. Because of this it is also interesting to consider that the values of the "PM_HIGH" for Beijing and Shenyang are 68.7% and 82.2% 0:s respectively, which is much lower than Guangzhu and Shanghai. This explains why the test score is higher than the train score and also why the Dummy accuracy is higher. Keep in mind that accuracy is just the percentage of correct predictions. Since the KMeansClassifier was trained to also predict the label 1 on a dataset with fewer 0:values it makes some sense why it would be worse than the dummy. Because of all of these facts, the accuracy score might not be the most interesting. There is also the fact that it is probably more important to be able to predict if the "PM_HIGH" is 1 and the air would be bad. So instead of just trying to maximize the accuracy score it might be better to try to minimize the number of false negatives while retaining as much accuracy as possible, when the prediction is that the air is good when it is in fact not. Perhaps it would be better to look at a confusion matrix to see the number of true positives, false positives, true negatives and false negatives. The confusion matrices for the dummy and the KMeansClassifier is shown below in figure 12.



(a) KMeans Classifier

(b) Dummy Classifier

Figure 12: Comparison of two confusion matrices

It can be seen that, as expected, the dummy simply predicts the majority class each time which makes it a useless model if the goal is to be able to predict if the air is bad or good. However, the KMeans classifier was also not good at all at predicting if the air is bad. With the configuration of the model shown in section 4, the classifier fails to classify most positive "HIGH_PM" values as positive which is bad. It is still better than the dummy though. To further investigate this, the F1 score of the models were compared. F1 score, according to [11] is a measurement for the predictive performance of a system and is defined as the harmonic mean between precision and recall that are given by these functions[11]:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1 score can be seen as a more interesting metric for this case since the datasets are heavily imbalanced, which makes the accuracy metric misleading and not very useful. And when comparing the F1 scores for the different models as shown below in figure 13 the usual trend of higher training accuracy than test accuracy becomes clear.

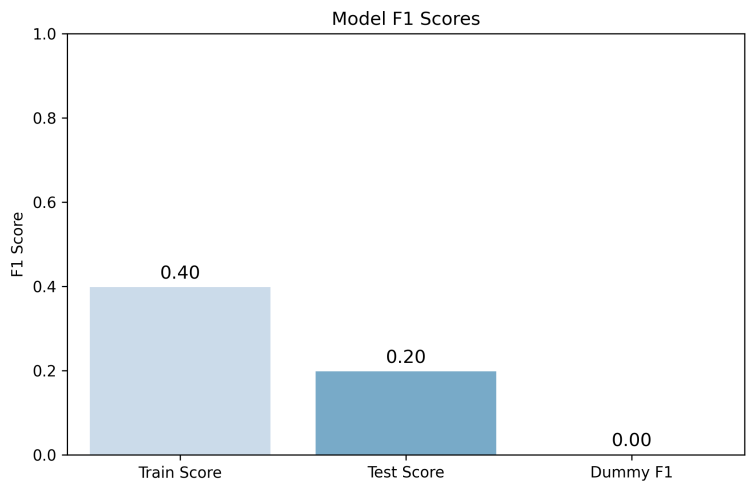


Figure 13: F1 Comparisons

These are not very good results, but at least the model had more predictive power than the dummy classifier. Experimentation with the `n_local_trials` parameter showed some promise and the test F1-score did go up to **22%**, but at the cost of accuracy.

6 Discussion

6.1 K-means suitability for this this assignment

K-means is an algorithm for unsupervised clustering while this assignment is about classification. In this regard, the algorithm itself is not made specifically for this assignment. Other more sophisticated algorithms

such as Support Vector Machines or an ensemble of decision trees could perform better. According to [9], decision tree based machine learning algorithms are dominating Kaggle competitions. For example, it could be attempted to use XGBoost or LightGBM(developed by Microsoft), to get better results in this classification task.

6.2 Difference between Train and Test Accuracy

Training accuracy is the accuracy the model gets when making predictions on the same data that it was trained on. Test accuracy is the accuracy of the model when making predictions for unseen data. The normal case is that the accuracy during training is higher than testing. This normal case is based on two assumptions that we made: That the model has the capacity to learn from the training data (sophisticated enough), and that the training and testing data has a similar distribution so it is meaningful to use the training data to fit a model that then makes prediction on the test data.

However in our implementation the model performed better on the testing data, which means the implementation is wrong, the result was random or that **at least one of the assumptions do not hold**. In this case, it is likely the case that the second assumption is violated, that the training and testing data are not from the same distribution. In fact, the training and testing are from different cities and the weather and pollution differ significantly between the cities as seen in the exploratory data analysis(EDA). To mitigate the issue, one suggestion would be to concatenate all data, shuffle it, and then split into training and testing to blend the distributions between training and testing data. And to improve the generalization capabilities of the model. Another way to mitigate this issue could be to include more features. As the model struggles to generalize between cities, most likely the features in themselves don't carry enough information to accurately predict the pollution. There might be other important features that are left out that explain the poor performance of the model.

Regarding the first assumption, that the k-means model is not sophisticated enough, this does not seem to be the case. If the number of clusters are increased to over five hundred, then the accuracy for the training data becomes higher than the testing data, thus it is able to overfit and "remember" a lot of information. And of course overfitting is a bad thing. However, the k-means algorithm cannot map latent relationships between features, making it a bit limited.

6.3 K-means Hyperparameters

A K-means algorithm has mainly two hyperparameters: number of clusters, and maximum number of iterations. Both are straight forward, the number of cluster specifies how many clusters the algorithm should classify, and the maximum number of iterations specifies when the algorithm should stop if it hasn't converged yet. In our implementation, a threshold for convergence has been added which can be helpful to stop the model from continuing without doing any changes and wasting resources.

One problem with the number of clusters to use was the imbalance of the data. If the number of clusters was set to two, the model only made the prediction 0. The algorithm works by assigning a label to each cluster by looking at what is the majority binary feature, i.e. if it mostly PM high or low. But because of the skewed distribution of the data, the majority is almost always PM low, leading to both clusters getting the label PM low. This was a huge issue when making predictions since all the predictions will be PM low. To mitigate this problem the number of clusters have to be increased, but increasing the number of clusters can also cause the model to fit to outliers or noise which makes the model worse. It is safe to say using this type of classifier for this problem may not be the perfect solution.

6.4 Evaluation Metric

Regarding how to define a successful model the metric is very important and depends heavily on the context and how the model will be used. A common metric such as accuracy usually is a simple and useful metric,

but in this case it's not perfect. Let's say that the model always predicts PM is low, for all the data in this assignment, then it would be correct 80% of the time, and thus get an accuracy of 0.8, which seems good, even though it's a terrible model since it cannot predict when the air quality is bad. Instead, a good way to measure the behavior of the model is with a confusion matrix, with true positives, false positives, etc. To get a better understanding of the confusion matrix, precision, and recall are important metrics. These metric were used in the evaluation of the model when the F1-score function was used.

Precision is how often the model is correct when it predicts a positive. Recall is how many of the actual positives the model captures [3].

In some contexts precision is super important, meaning that if something is predicted to be positive, then it actually is positive. An example is for medical screening where every time the model predicts the presence of a disease, an invasive and expensive operation must be carried out. If the model often wrongly predicts the presence of a disease a lot of unnecessary and expensive operations will be carried out. In this case the model might cause more harm than good. However, in other cases, it might be very important with recall, that the model finds all the true positives. An example can be a fraud detection system that predicts fraudulent transactions. If the model predicts a fraudulent transaction, the cost to check if it actually is fraudulent might be very low, and the cost of missing one fraudulent transaction might be very high. This type of model should be tuned to not miss any true positives. And in the case of predicting "PM_HIGH", it might be more important to not have a lot of false negatives.

So to summarize, the metric of evaluation should be chosen in a way that maximized the "good" of the model, while minimizing the "bad". What this means in each context is different.

With this said, this model should prioritize recall. It is important that the model predicts most days with high PM correctly, even though it might wrongly predict high PM sometimes. A high PM prediction might mean that people put on a face mask, which is not a big effort. However if the model predicts low PM, while it actually is high PM, then people that are at risk, old and children, might go outside without face mask, causing harm to their health.

Another thing with the evaluation, is that it is pretty unstable. Because the initial points of the k-means are initialized randomly, it often converges differently also. This can mean that it is difficult to compare results. In order to mitigate this issue, KMeans ++ initialization was implemented to try and get better results.

7 Suggestions for further experimentation

7.1 Principal Component Analysis

To further improve the performance of the model, Principle Component Analysis could perhaps be used to reduce the dimension of the feature space. This might be good because K-means algorithms perform better in low dimensionality [1].

It would also be interesting to do a dimensionality reduction and plot the result of it to see how different the train and test set actually are, it could also give some hints as to why the performance of the model is not very good in this case.

7.2 Cluster Label Assignment

Currently, majority voting is used to assign the labels to the clusters. So, if more than half of all points in a cluster have the label 1, the cluster will be assigned the label 1. A small code snippet below shows this procedure.

```
val = num_ones /( num_ones + num_zeros )
threshold = 0.5
```

```
    if val >= threshold:
        # Assign label 1
```

It would probably be a good idea to work on changing this procedure. Different values of the threshold have been attempted and some did show some improvements to the F1 test score but at the cost of a drop in accuracy. Perhaps some more advanced methods could be experimented with to better assign the clusters the optimal labels in terms of having the smallest number of false negative predictions while also having good accuracy.

7.3 Other models

As briefly mentioned in section 6.1, a KMeans classifier might not be the most suitable model for the task of this assignment. As mentioned, XGBoost or LightGBM and other decision tree based algorithms might be better and would potentially get better results. These models could be experimented with to try and get better results.

8 Appendix - Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import f1_score, accuracy_score, confusion_matrix,
    ConfusionMatrixDisplay
from sklearn.model_selection import cross_val_score
from sklearn.base import BaseEstimator
from copy import deepcopy
```

A Reading and Cleaning data

```
#READING DATA
beijing_df = pd.read_csv('Cities/Beijing_labeled.csv')
chengdu_df = pd.read_csv('Cities/Chengdu_labeled.csv')
guangzhou_df = pd.read_csv('Cities/Guangzhou_labeled.csv')
shanghai_df = pd.read_csv('Cities/Shanghai_labeled.csv')
shenyang_df = pd.read_csv('Cities/Shenyang_labeled.csv')
```

```
#RENAMING COLUMNS
rename_dict = {
    'season': 'Season',
    'DEWP': 'Dew_Point',
    'HUMI': 'Humidity',
    'PRES': 'Pressure',
    'TEMP': 'Temperature',
    'Iws': 'Wind_Speed',
    'precipitation': 'Precipitation',
    'cbwd_NE': 'Wind_Dir_NE',
    'cbwd_NW': 'Wind_Dir_NW',
    'cbwd_SE': 'Wind_Dir_SE'
}

dfs = [beijing_df, chengdu_df, guangzhou_df, shanghai_df, shenyang_df]

for df in dfs:
    df.rename(columns=rename_dict, inplace=True)

beijing_df.head()
```

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

```
all_cities_data = [beijing_df, chengdu_df, guangzhou_df, shanghai_df, shenyang_df]
cities = ["Beijing", "Chengdu", "Guangzhou", "Shanghai", "Shenyang"]
```

```
all_df = pd.concat(all_cities_data, keys=cities, names=["City"], ignore_index=
    False).reset_index(0).reset_index(drop=True)
# print(all_df)
```

B Exploratory Data Analysis

```
stats = all_df.describe()
# print(stats)
```

```
plt.figure(figsize=(10, 8))
blue = sns.color_palette("Blues", n_colors=4)
sns.boxplot(x='Season', y='Temperature', data=all_df, palette=blue)
plt.title('Temperature Distribution by Season')
plt.xlabel('Season')
plt.ylabel('Temperature ( C )')

plt.show()
```

```
blue_shades = sns.color_palette("Blues", n_colors=5)

plt.figure(figsize=(10, 6))
sns.boxplot(x='Season', y='Temperature', hue='City', data=all_df, palette=
    blue_shades)
plt.title('Temperature by Season and City')
plt.show()
```

```
plt.figure(figsize=(8,6))
sns.countplot(x='Season', hue='PM_HIGH', data=all_df, palette='Blues')
plt.title('PM_HIGH Count by Season')
plt.show()
```

```
distribution = all_df['PM_HIGH'].value_counts()

# Calculate sparsity
total_elements = sum(distribution)
high_pm = distribution[0]
sparsity = high_pm / total_elements
print(f"Sparsity: {sparsity:.3f}")

# Plot
labels = ['Low PM', 'High PM']
sizes = [high_pm, total_elements - high_pm]

plt.figure(figsize=(8, 8))
wedges, texts, autotexts = plt.pie(
    sizes,
    labels=labels,
    autopct='%1.1f%%',
    startangle=0,
    colors=['#dce7ff', '#000080'],
    wedgeprops={'edgecolor': 'black', 'linewidth': 0.5},
    explode=[0, 0.1],        # Slightly pull out the small slice
    pctdistance=1.15,         # Move percentage text away from center
    labeldistance=1.32        # Move labels further out
```

```
)

for autotext in autotexts:
    autotext.set_fontsize(16)

plt.title('Distribution of PM, All Cities', fontsize=16)
plt.show()
```

```
Sparsity: 0.800
```

```
distribution_beijing = beijing_df['PM_HIGH'].value_counts()
total_elements_beijing = sum(distribution_beijing)
high_pm_beijing = distribution_beijing[0]
lowpm_beijing = high_pm_beijing / total_elements_beijing
print(f"Low PM Beijing: {lowpm_beijing:.3f}")

distribution_chengdu = chengdu_df['PM_HIGH'].value_counts()
total_elements_chengdu = sum(distribution_chengdu)
high_pm_chengdu = distribution_chengdu[0]
lowpm_chengdu = high_pm_chengdu / total_elements_chengdu
print(f"Low PM Chengdu: {lowpm_chengdu:.3f}")

distribution_guangzhou = guangzhou_df['PM_HIGH'].value_counts()
total_elements_guangzhou = sum(distribution_guangzhou)
high_pm_guangzhou = distribution_guangzhou[0]
lowpm_guangzhou = high_pm_guangzhou / total_elements_guangzhou
print(f"Low PM Guangzhu: {lowpm_guangzhou:.3f}")

distribution_shanghai = shanghai_df['PM_HIGH'].value_counts()
total_elements_shanghai = sum(distribution_shanghai)
high_pm_shanghai = distribution_shanghai[0]
lowpm_shanghai = high_pm_shanghai / total_elements_shanghai
print(f"Low PM Shanghai: {lowpm_shanghai:.3f}")

distribution_shenyang = shenyang_df['PM_HIGH'].value_counts()
total_elements_shenyang = sum(distribution_shenyang)
high_pm_shenyang = distribution_shenyang[0]
lowpm_shenyang = high_pm_shenyang / total_elements_shenyang
print(f"Low PM Shenyang: {lowpm_shenyang:.3f}")
```

```
Low PM Beijing: 0.687
Low PM Chengdu: 0.705
Low PM Guangzhu: 0.936
Low PM Shanghai: 0.902
Low PM Shenyang: 0.822
```

```
corr_matrix = all_df.corr(numeric_only=True)
plt.figure(figsize=(14,12))
sns.heatmap(corr_matrix, annot=True, cmap='Blues')
plt.title("Feature Correlation Matrix, All Cities")
plt.tight_layout()
plt.show()
```

C The classifier implemented for this assignment

```python
from sklearn.cluster import KMeans
class KMeansClassifier(BaseEstimator):
    """
    Implementation of a KMeans Classifier for assignment 3 in Design of AI Systems.

    Some highlights:
        - 2 centroid initialization methods: random and KMeans ++
        - hyperparameters:
            - number of clusters
            - centroid initialization
            - max number of iterations
            - random state
            - n local trials (only if KMeans ++ initialization)
        - fit, predict and score functions implemented
    """


    _parameter_constraints: dict = {
        "centroid_initialisation_options": ["k-means++", "random"],
        "number_of_clusters_minimum": 1,
        "convergence_threshold_range": [0,1]
    }


    def __init__(self, number_of_clusters: int = 8, centroid_initialisation: str = "k-means++", max_iterations: int = 100, random_state: int = None, n_local_trials: int = None, default_scoring_function: callable = accuracy_score, convergence_threshold: float = 0.02):
            self.number_of_clusters: int = number_of_clusters
            self.centroid_initialisation: str = centroid_initialisation
            self.max_iterations: int = max_iterations
            self.random_state: int = random_state
            self.random: np.random.RandomState = np.random.RandomState(random_state)
            self.n_local_trials: int = n_local_trials
            self.default_scoring_function: callable = default_scoring_function
            self.convergence_threshold = convergence_threshold

            self._centroids: np.ndarray = None
            self.cluster_labels: dict = {}

            KMeansClassifier._check_centroid_initialisation(centroid_initialisation)
            KMeansClassifier._check_number_of_clusters(number_of_clusters)
            KMeansClassifier._check_convergence_threshold(convergence_threshold)

    @classmethod
    def _check_centroid_initialisation(cls, centroid_init: str) -> None:
        """
        Does some checks on the centroid_initialisation hyperparemeter.
        """
        constraints = KMeansClassifier._parameter_constraints
        centroid_init_options = constraints["centroid_initialisation_options"]
        if centroid_init not in centroid_init_options:
```

```python
            raise ValueError(f"Invalid value of centroid_initialisation: '{centroid_init}'. Expected one of the following: {centroid_init_options}.")

    @classmethod
    def _check_number_of_clusters(cls, num_clusters: int) -> None:
        """
        Does some checks on the centroid_initialisation hyperparemeter.
        """
        constraints = KMeansClassifier._parameter_constraints
        number_of_clusters_minimum = constraints["number_of_clusters_minimum"]
        if num_clusters < number_of_clusters_minimum :
            raise ValueError(f"Invalid value of number_of_clusters: '{num_clusters}'. Value has to be greater than {number_of_clusters_minimum}.")

    @classmethod
    def _check_convergence_threshold(cls, convergence_threshold: float) -> None:
        """
        Does some checks on the centroid_initialisation hyperparemeter.
        """
        constraints = KMeansClassifier._parameter_constraints
        convergence_threshold_range = constraints["convergence_threshold_range"]
        lower = convergence_threshold_range[0]
        upper = convergence_threshold_range[1]
        if lower > convergence_threshold > upper :
            raise ValueError(f"Invalid value of convergence_threshold: '{convergence_threshold}'. Value has to be in the range: {convergence_threshold_range}.")


    def get_centroids(self):
        """
        Returns the centroids of the classifier.
        """
        return self._centroids


    def _get_initial_centroids_Kmeans_plusplus(self, X: pd.DataFrame) -> np.ndarray:
        """
        Uses a simplified version of the Scikit-learn implementation of KMeans ++ initialization of the centroids.

        Args:
            X: `pd.DataFrame` = The input training features

        Returns:
            `np.ndarray`: The centroids.
        """
        n_clusters = self.number_of_clusters
        _, n_features = X.shape

        # Empty list for the centroids
        centroids = np.empty((n_clusters, n_features), dtype=X.dtypes)

        # Initialise first random centroid
        first_centroid = X.sample(1, random_state=self.random_state).values
        centroids[0] = first_centroid

```

```python
        # Squared distances to the first centroid for all datapoints
        squared_distances = np.sqrt(np.sum((first_centroid - X) ** 2, axis=1))
        current_potential = squared_distances.sum()

        # Defining the number of local seeding trials
        # In the code of Scikit-learn library implementation of Kmeans this is used when other value is not specified.
        # Apparently it is supposed to help the result and the source code refers to Arthur/Vassilvitskii for more information
        if self.n_local_trials is None:
            self.n_local_trials = 2 + int(np.log(n_clusters))

        # Selecting the remaining n_clusters - 1 centroids
        for c in range(1, n_clusters):
            # Below code also heavily inspired from Scikit-learn library implementation of Kmeans
            # It generates n_local_trials random values between 0 and 1 that are scaled by the current potential of the distances
            random_values = self.random.rand(self.n_local_trials) * current_potential

            # Finding the indexes where in the cumulative sum the random values would fit to still have the same order. Those will be the candidate centroids.
            candiate_centroid_indexes = np.searchsorted(np.cumsum(squared_distances), random_values)
            candidate_centroids = X.values[candiate_centroid_indexes]

            distance_to_candidates = np.array([np.sum((candidate_centroid - X) ** 2, axis=1) for candidate_centroid in candidate_centroids])

            # Returns the element wise minimum, [1, 5, 3] and [5, 2 ,1] would -> [1, 2, 1]
            np.minimum(squared_distances, distance_to_candidates, out=distance_to_candidates)
            candidates_potential = distance_to_candidates.sum(axis=1)

            # Decide which candidate is the best
            best_candidate_index = np.argmin(candidates_potential)
            current_potential = candidates_potential[best_candidate_index]

            # Update squared distances
            squared_distances = distance_to_candidates[best_candidate_index]

            best_candidate = candidate_centroids[best_candidate_index]

            # Add the centroid to the array of centroids
            centroids[c] = best_candidate

        return np.array(centroids, dtype=np.float64) # Had to specify the type here for some reason


    def _get_initial_centroids_random(self, X: pd.DataFrame) -> np.ndarray:
        """
        Randomly samples the centroids from the points in the data.

        Args:
            X: `pd.DataFrame` = The input training features
```

```python

        Returns:
            `np.ndarray`: The centroids.
        """
        return (X.sample(n=self.number_of_clusters, random_state=self.random_state)).to_numpy()


    def fit(self, X: pd.DataFrame, y: pd.Series) -> None:
        """
        Fits the KMeans model to the data and prepares it for making predictions.

        Args:
            X: `pd.DataFrame` = The input training features
            y: `pd.Series` = The input training labels

        Returns:
            None: Changes the object so it is prepared to make predictions.
        """

        # Initialising the centroids
        if self.centroid_initialisation == "k-means++":
            self._centroids = self._get_initial_centroids_Kmeans_plusplus(X)
        else:
            self._centroids = self._get_initial_centroids_random(X)

        # Transforming to numpy
        X = X.to_numpy()

        # Keeping track of the old centroids to check for covergence.
        old_centroids = deepcopy(self._centroids)
        def compute_centroid_minimum_distance(old_center, new_center):
            distances_between_centroids = np.sqrt(np.sum((old_center[:, np.newaxis] - new_center) ** 2, axis=2))
            # Interestingly enough, when computing this above the diagonal should always be 0, but it is not which ruins the calculations.
            # [[9.91430585e-02 1.98624831e+02]
            # [1.96331650e+02 2.25158895e+00]], one example with two centroids. The diagonals are supposed to be 0 but they are not.
            # Therefore the diagonal had to be filled in a new way opposed to doing the following: distances_between_centroids[distances_between_centroids = 0]
            np.fill_diagonal(distances_between_centroids, np.inf)
            return np.min(distances_between_centroids) # Value is 0 when comparing distance with itself

        old_minimum_distance_between_centroids = compute_centroid_minimum_distance(self._centroids, self._centroids)

        for i in range(self.max_iterations):
            # Distances where each row will be a certain datapoint and the column will be a centroid, the values are the distances from that centroid.
            # np.newaxis is needed here to transform X from the shape (2071, 10) to (2071, 1, 10). This is needed since there are multiple centroids.
            # print(X.shape) # (2071, 10)
            # print(X[:, np.newaxis].shape) # (2071, 1, 10)
```

```
            distances = self._euclidean_distances(self._centroids, X[:, np.newaxis], axis=2)

            # Assigning each datapoint to the centroid that minimizes the distance, looking at each row for each datapoint
            new_labels = np.argmin(distances, axis=1)

            # Function for finding the mean of a cluster
            mean_of_cluster = lambda cluster: X[new_labels == cluster].mean(axis=0)
            mean_of_clusters = [mean_of_cluster(i) for i in range(self.number_of_clusters)]
            self._centroids = np.array(mean_of_clusters)

            # Convergence
            centroid_movement = np.sqrt(np.sum((old_centroids - self._centroids) ** 2, axis=1))
            max_centroid_movement = np.max(centroid_movement)
            if max_centroid_movement < self.convergence_threshold * old_minimum_distance_between_centroids:
                break
            else:
                old_centroids = deepcopy(self._centroids)
                old_minimum_distance_between_centroids = compute_centroid_minimum_distance(old_centroids, self._centroids)


        # Computing the final distances to the centroid and assigning each point to a cluster
        distances = self._euclidean_distances(self._centroids, X[:, np.newaxis], axis=2)
        cluster_assignments = np.argmin(distances, axis=1)

        for i in range(self.number_of_clusters):
            counts = y[cluster_assignments == i].value_counts()
            num_zeros = counts.get(0, 0)
            num_ones = counts.get(1, 0)

            val = num_ones /(num_ones + num_zeros)

            # Experimented a bit with this value but did not make much difference:
            threshold = 0.5
            if val >= threshold:
                self.cluster_labels[i] = 1
            else:
                self.cluster_labels[i] = 0


    def predict(self, x: pd.Series):
        """
        Predicts the label of a point.

        Args:
            x: The point to predict the label of.

        Returns:
            int: The predicted label for the point.
        """
```

```python
        x = x.to_numpy()
        # Distances to all centroids
        dist = self._euclidean_distances(x, self._centroids, axis=1)
        # Finding the centroid that with the shortest distance
        closest_cluster = np.argmin(dist)
        return self.cluster_labels[closest_cluster]


    def score(self, X: pd.DataFrame, y: pd.Series, score_function: callable = None, show: bool = False) -> float:
        """
        Calculates the score of the `KMeansClassifier`.

        Args:
            X: `pd.DataFrame` = The features.
            y: `pd.Series` = The true labels.
            scrore_function: `callable` = The score function to use for scoring.
            show: `bool` = If the score should be printed or not.

        Returns:
            `float`: The score of the Classifier
        """
        true_values = []
        predictions = []
        for i in range(len(X)):
            predictions.append(self.predict(X.iloc[i]))
            true_values.append(y.iloc[i])

        # Making sure there is a scoring function
        if score_function is None:
            score_function = self.default_scoring_function

        score_value = score_function(true_values, predictions)
        if show:
            print(f"{score_function.__name__}: {score_value}")

        return score_value


    def _euclidean_distances(self, point, data, axis: int = 1) -> np.ndarray:
        """
        Function for calculating the euclidean distances from a point to a bunch of other points. Inspired by:
        https://towardsdatascience.com/create-your-own-k-means-clustering-algorithm-in-python-d7d4c9077670/

        Args:
            point: The point or points to calculate the distances from
            data: The other datapoints the distance should be calcualted to for each point
            axis: How to

        Returns:
            np.ndarray: The array of all the distances.
        """
        return np.sqrt(np.sum((point - data)**2, axis=axis))
```

D Applying the Classifier

In this step the goal is to train and validate our system on the cities Beijing ans Shenyang. Next the classifier should be separately evaluated on the cities Guangzhou and Shanghai.

D.1 Training and Validation

D.1.1 Creating the training set

```
training_and_validation_set = pd.concat([beijing_df, shenyang_df]).reset_index(
    drop=True)
X_train = training_and_validation_set.drop(['PM_HIGH'], axis=1)
y_train = training_and_validation_set['PM_HIGH']
```

D.1.2 Which method for initializing the centroids should be used

```
def get_mean_score(model: KMeansClassifier):
    return np.mean(cross_val_score(model, X_train, y_train))


k_values = [k for k in range(1, 41)]
distortions = []
plusplus = []
random = []
for i in k_values:
    classifier_plusplus = KMeansClassifier(number_of_clusters=i, max_iterations
        =1000, centroid_initialisation="k-means++", n_local_trials=50,
        default_scoring_function=accuracy_score)
    classifier_random = KMeansClassifier(number_of_clusters=i, max_iterations
        =1000, centroid_initialisation="random", default_scoring_function=
        accuracy_score)
    plusplus.append(get_mean_score(classifier_plusplus))
    random.append(get_mean_score(classifier_random))

plt.plot(k_values, plusplus, 'rx-', label="KMeans++")
plt.plot(k_values, random, 'bx-', label="Random")
plt.legend()
plt.xlabel('Number of Clusters/Centroids')
plt.ylabel('Mean cross valdation Accuracy score')
plt.title('Cross validation Accuracy scores')
plt.grid()
plt.tight_layout()
plt.savefig("Pics/random_vs_plusplus.png", dpi=300)
plt.show()
```

D.1.3 Cross validation for different values of K and the Elbow method

```
from scipy.spatial.distance import cdist

def calc_distortions(model: KMeansClassifier, X: pd.DataFrame):
    """
    Calculating the average square distance from each point to its cluster center.
        Used for the elbow method. See
```

```
    https://www.geeksforgeeks.org/elbow-method-for-optimal-value-of-k-in-kmeans/ for more info. Function is taken from that source.
    """
    return sum(np.min(cdist(X, model.get_centroids(), 'euclidean'), axis=1)**2) / X.shape[0]


k_values = [k for k in range(1, 101)]
distortions = []
cv_accuracy_scores = []
for i in k_values:
    classifier = KMeansClassifier(number_of_clusters=i, max_iterations=1000, default_scoring_function=accuracy_score, random_state=42)
    cv_accuracy_scores.append(get_mean_score(classifier))
    classifier.fit(X_train, y_train)
    distortions.append(calc_distortions(classifier, X_train))

fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(12, 4))

ax1.plot(k_values, distortions, 'bx-')
ax1.set_xlabel('Number of Clusters/Centroids')
ax1.set_ylabel('Distortion')
ax1.set_title('The Elbow Method')
ax1.grid()

ax2.plot(k_values, cv_accuracy_scores, 'bx-')
ax2.set_xlabel('Number of Clusters/Centroids')
ax2.set_ylabel('Mean cross valdation accuracy score')
ax2.set_title('Cross validation accuracy scores')
ax2.grid()


fig.tight_layout()
plt.savefig("Pics/elbow_cvaccuracy.png", dpi=300)
plt.show()
```

D.1.4 Cross validation F1 scores

```
k_values = [k for k in range(1, 41)]
cv_f1_scores = []
for i in k_values:
    classifier = KMeansClassifier(number_of_clusters=i, max_iterations=1000, default_scoring_function=f1_score)
    cv_f1_scores.append(get_mean_score(classifier))



plt.plot(k_values, cv_f1_scores, 'bx-')
plt.xlabel('Number of Clusters/Centroids')
plt.ylabel('Mean cross valdation F1 score')
plt.title('Cross validation F1 scores')
plt.grid()
plt.tight_layout()
plt.savefig("Pics/cvf1.png", dpi=300)
plt.show()
```

D.1.5 Final training with number_of_clusters = 8

```
classifier = KMeansClassifier(number_of_clusters=8,
                              centroid_initialisation="k-means++",
                              max_iterations=1000,
                              random_state=42,
                              default_scoring_function=accuracy_score)

classifier.fit(X_train, y_train)

from sklearn.dummy import DummyClassifier
scikit_dummy = DummyClassifier(strategy='most_frequent')
scikit_dummy.fit(X_train, y_train)
```

D.2 Evaluation

```
# Creating X_test and y_test
evaluation_set = pd.concat([guangzhou_df, shanghai_df]).reset_index(drop=True)
X_test = evaluation_set.drop(['PM_HIGH'], axis=1)
y_test = evaluation_set['PM_HIGH']
```

Evaluation of KMeans Classifier

```
# Evaluation of KMeans classifier
print(f"KMeans classifier training accuracy: {classifier.score(X_train, y_train)}"
    )
print(f"KMeans classifier test accuracy: {classifier.score(X_test, y_test)}")

# Getting the predictions
true_values = []
predictions = []

for i in range(len(X_test)):
    predictions.append(classifier.predict(X_test.iloc[i]))
    true_values.append(y_test.iloc[i])

cm_clf = confusion_matrix(true_values, predictions)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_clf,display_labels=[0, 1])
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix KMeans classifier predictions")
plt.tight_layout()
plt.savefig("Pics/confusion_clf.png", dpi=300)
plt.show()
```

```
KMeans classifier training accuracy: 0.7575129533678756
KMeans classifier test accuracy: 0.8830928597854236
```

Evaluation of Dummy Classifier

```
pred = scikit_dummy.predict(X_test)
print(f"Dummy classifier test accuracy: {accuracy_score(y_test, pred)}")

cm_dummy = confusion_matrix(true_values, pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_dummy,display_labels=[0, 1])
disp.plot(cmap=plt.cm.Blues)
```

```
plt.title("Confusion Matrix Dummy classifier predictions")
plt.tight_layout()
plt.savefig("Pics/confusion_dummy.png", dpi=300)
plt.show()
```

```
Dummy classifier test accuracy: 0.9189789123196448
```

D.2.1 Comparing accuracy scores

```
import matplotlib.pyplot as plt

train_score = classifier.score(X_train, y_train)
val_score = np.mean(cross_val_score(classifier, X_train, y_train))
test_score = classifier.score(X_test, y_test)
dummy_accuracy = accuracy_score(y_test, scikit_dummy.predict(X_test))
values = [train_score, val_score, test_score, dummy_accuracy]

# Plotting
plt.figure(figsize=(8, 5))
colors = sns.color_palette("Blues", 4)
sns.barplot(x=["Train Score","Validation Score(Mean)", "Test Score", "Dummy Accuracy"], y=values, palette=colors)
plt.ylim(0, 1)
plt.ylabel("Accuracy")
plt.title("Model Accuracy Scores")

# Display the values on top of bars
for i, v in enumerate(values):
    plt.text(i, v + 0.02, f"{v:.2f}", ha="center", fontsize=12)

plt.savefig("Pics/comparison.png", dpi=300)
plt.show()
```

D.2.2 Comparing F1-scores

```
train_score = classifier.score(X_train, y_train, score_function=f1_score)
test_score = classifier.score(X_test, y_test, f1_score)
dummy_accuracy = f1_score(y_test, scikit_dummy.predict(X_test))
values = [train_score, test_score, dummy_accuracy]

# Plotting
plt.figure(figsize=(8, 5))
colors = sns.color_palette("Blues", 3)
sns.barplot(x=["Train Score", "Test Score", "Dummy F1"], y=values, palette=colors)
plt.ylim(0, 1)
plt.ylabel("F1 Score")
plt.title("Model F1 Scores")

# Display the values on top of bars
for i, v in enumerate(values):
    plt.text(i, v + 0.02, f"{v:.2f}", ha="center", fontsize=12)

plt.savefig("Pics/comparison_f1.png", dpi=300)
plt.show()
```

References

[1] Carly L. Clayman, Satish M. Srinivasan, and Raghvinder S. Sangwan. "K-means Clustering and Principal Components Analysis of Microarray Data of L1000 Landmark Genes". In: *Procedia Computer Science* 168 (2020), pp. 97–104. ISSN: 1877-0509. DOI: 10.1016/j.procs.2020.02.265. URL: https://www.sciencedirect.com/science/article/pii/S187705092030404X.

[2] GeeksforGeeks. *Elbow Method for Optimal Value of k in KMeans*. Accessed: 2025-02-09. 2021. URL: https://www.geeksforgeeks.org/elbow-method-for-optimal-value-of-k-in-kmeans/.

[3] Google Developers. *Classification: Accuracy, Recall, Precision, and Related Metrics*. https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall. Accessed: February 05, 2025. n.d.

[4] IBM. *K-Means Cluster Analysis - Convergence Criteria*. Accessed: 2025-02-09. 2025. URL: https://www.ibm.com/docs/en/spss-statistics/beta?topic=analysis-k-means-cluster-convergence-criteria.

[5] Eda Kavlakoglu and Vanna Winland. "What is k-means clustering?" In: *IBM* (2024). URL: https://www.ibm.com/think/topics/k-means-clustering.

[6] Chris (NHC) Landsea. *How are Atlantic hurricanes ranked? The Saffir-Simpson Scale*. https://www.smosstorm.org/FAQ/How-are-Atlantic-hurricanes-ranked.-The-Saffir-Simpson-Scale. Contributed by Chris Landsea (NHC). n.d. (Visited on 02/04/2025).

[7] UCI Machine Learning and 1 collaborator. *PM2.5 Data of Five Chinese Cities*. https://www.kaggle.com/datasets/uciml/pm25-data-for-five-chinese-cities/data. Measurements for Shenyang, Chengdu, Beijing, Guangzhou, and Shanghai. PM2.5 refers to atmospheric particulate matter with a diameter less than 2.5 micrometers, used as a measure of pollution. n.d. (Visited on 02/04/2025).

[8] Nitin R. Londhe, Dipti S. Doye, and Ramesh B. Deshmukh. "Subset K-Means Approach for Handling Imbalanced-Distributed Data". In: *Emerging ICT for Bridging the Future - Proceedings of the 49th Annual Convention of the Computer Society of India (CSI), Volume 2*. Springer, 2014, pp. 497–508. DOI: 10.1007/978-3-319-13731-5_54. URL: https://link.springer.com/chapter/10.1007/978-3-319-13731-5_54.

[9] Prashant. *LightGBM Classifier in Python*. https://www.kaggle.com/code/prashant111/lightgbm-classifier-in-python. Accessed: 2025-02-11. 2021.

[10] D. Sculley et al. "Hidden Technical Debt in Machine Learning Systems". In: *Advances in Neural Information Processing Systems*. 2015, pp. 2503–2511. URL: https://papers.nips.cc/paper_files/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html.

[11] Wikipedia contributors. *F-score — Wikipedia, The Free Encyclopedia*. [Online; accessed February 10, 2025]. 2024. URL: https://en.wikipedia.org/wiki/F-score.